

Endless Runner — Vibe Coding Chat History (Tasks 1–3)

Repository: SETAPESU26/39_endless-runner

This PDF contains the Copilot/LLM interaction provided for Tasks 1–3, in order.

Source transcript 1

I'll inspect the current Game Over flow and main loop first, then fit difficulty selection into the existing engine rather than

Ran terminal command: git status --short; git diff -- main.py game/game_engine.py

Read

Read

Read

The current flow already keeps one engine instance running, but any key after Game Over sets `quit\_requested`, so replay

Generating patch (77 lines) in

Generating patch (79 lines) in

```
...
...

```

```
Ran terminal command: $env:PYTHONDONTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle
```

```
pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0
    assert not engine.game_over and not engine.quit_requested
```

```
engine.game_over = True
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over
```

```
texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface
```

```
class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
```

```

    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
#@ | python -
Completed with input: $env:PYTHONDONTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0
    assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
#@ | python -

```

The reset and selection assertions passed for Easy, Medium, Hard, and Exit. The render check caught one local layout def

Generating patch (9 lines) in
 ...

...

```
Ran terminal command: $env:PYTHONDOTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0
    assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
#@ | python -
Completed with input: $env:PYTHONDOTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
```

```

        (pygame.K_1, "Easy", 4, 85),
        (pygame.K_2, "Medium", 6, 70),
        (pygame.K_3, "Hard", 8, 55),
    ):
        engine.game_over = True
        engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
        assert engine.difficulty == name
        assert (engine.speed, engine.spawn_interval) == (speed, interval)
        assert engine.score == 0 and engine.obstacles == []
        assert engine.player.y == engine.ground_y - engine.player.height
        assert engine.player.vy == 0 and engine.player.on_ground
        assert engine._spawn_timer == 0 and engine.distance == 0
        assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
#@ | python -

```

The rerun exposed a test-fixture mistake rather than a code failure: replay correctly reset the score to zero, but the r

Ran terminal command: \$env:PYTHONDONTWRITEBYTECODE = '1'; \$env:SDL_VIDEODRIVER = 'dummy'; @'

```

import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0
    assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.score = 9
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))

```

```

assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
'@ | python -

```

All three difficulty choices now reset position, jump state, score, obstacles, timer, and distance; the settings are dis

```

Ran terminal command: $env:PYTHONDONTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

```

```

pygame.init()
engine = GameEngine(800, 400)
engine.score = 3
engine.obstacles = [Obstacle(160, engine.ground_y, 120)]
engine.update()
assert engine.game_over, "Swept collision did not reach Game Over"
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_3))
assert not engine.game_over and engine.difficulty == "Hard"
assert engine.speed == 8 and engine.spawn_interval == 55
assert engine.score == 0 and not engine.obstacles
engine.update()
assert not engine.game_over and engine.speed > 8
assert engine.score == 0 and not engine.obstacles
print("PASS: actual collision -> Hard selection -> fresh running game")
pygame.quit()
'@ | python -

```

The end-to-end check passed from a real swept collision through selecting Hard and resuming a fresh run. I'm checking sy

```

Ran terminal command: @'
from pathlib import Path
for filename in ("main.py", "game/game_engine.py"):
    source = Path(filename).read_text(encoding="utf-8")
    compile(source, filename, "exec")
print("PASS: Python syntax/compile checks for main.py and game/game_engine.py")
'@ | python -

```

```
Ran terminal command: git diff -- main.py game/game_engine.py; git status --short
```

```
Checked , , no problems found
```

Task 3 is implemented. The Game Over screen now offers `1 - Easy`, `2 - Medium`, `3 - Hard`, and `4 - Exit`. Easy, Medium

Validation passed: an actual collision led to Game Over, selecting Hard started a fresh running game, all three settings

****Files:**** This Task 3 change modified `game\_engine.py`. `main.py` remains modified from Task 2 and provides the exit ha

Made changes.

Source transcript 2

I'll inspect the current Game Over flow and main loop first, then fit difficulty selection into the existing engine rath

Ran terminal command: git status --short; git diff -- main.py game/game_engine.py

Read

Read

Read

The current flow already keeps one engine instance running, but any key after Game Over sets `quit\_requested`, so replay

Generating patch (77 lines) in

Generating patch (79 lines) in

...
...

Ran terminal command: \$env:PYTHONDONTWRITEBYTECODE = '1'; \$env:SDL_VIDEODRIVER = 'dummy'; @'

```
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0
    assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
'@ | python -
```

```

Completed with input: $env:PYTHONDOTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0
    assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
#@ | python -

```

The reset and selection assertions passed for Easy, Medium, Hard, and Exit. The render check caught one local layout def

Generating patch (9 lines) in

```

...
...

```

```

Ran terminal command: $env:PYTHONDOTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9

```

```

engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0
    assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
#@ | python -
Completed with input: $env:PYTHONDONTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0

```

```

        assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:
    def __init__(self):
        self.placements = []
    def fill(self, color):
        pass
    def blit(self, surface, destination):
        self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
'@ | python -

```

The rerun exposed a test-fixture mistake rather than a code failure: replay correctly reset the score to zero, but the n

Ran terminal command: \$env:PYTHONDONTWRITEBYTECODE = '1'; \$env:SDL_VIDEODRIVER = 'dummy'; @'

```

import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.game_over = True
engine.score = 9
engine.obstacles = [Obstacle(300, engine.ground_y, 6)]
engine.player.y -= 40
engine.player.vy = -15
engine.player.on_ground = False
engine._spawn_timer = 23
engine.distance = 120

for key, name, speed, interval in (
    (pygame.K_1, "Easy", 4, 85),
    (pygame.K_2, "Medium", 6, 70),
    (pygame.K_3, "Hard", 8, 55),
):
    engine.game_over = True
    engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=key))
    assert engine.difficulty == name
    assert (engine.speed, engine.spawn_interval) == (speed, interval)
    assert engine.score == 0 and engine.obstacles == []
    assert engine.player.y == engine.ground_y - engine.player.height
    assert engine.player.vy == 0 and engine.player.on_ground
    assert engine._spawn_timer == 0 and engine.distance == 0
    assert not engine.game_over and not engine.quit_requested

engine.game_over = True
engine.score = 9
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_4))
assert engine.quit_requested and engine.game_over

texts = []
class RecordingFont:
    def __init__(self, font):
        self.font = font
    def render(self, text, antialias, color):
        surface = self.font.render(text, antialias, color)
        texts.append((text, surface))
        return surface

class RecordingScreen:

```

```

def __init__(self):
    self.placements = []
def fill(self, color):
    pass
def blit(self, surface, destination):
    self.placements.append((surface, destination.copy()))

engine.game_over_title_font = RecordingFont(pygame.font.SysFont("Arial", 48))
engine.game_over_message_font = RecordingFont(pygame.font.SysFont("Arial", 24))
screen = RecordingScreen()
engine.render(screen)
labels = [text for text, _ in texts]
assert "Final Score: 9" in labels
assert all(f"{index} - {option}" in labels for index, option in enumerate(("Easy", "Medium", "Hard", "Exit"), 1))
centers = {id(surface): rect.centery for surface, rect in screen.placements}
assert centers[id(texts[0][1])] != centers[id(texts[1][1])], "Game Over title overlaps final score"
print("PASS: difficulty reset, menu render, and Exit selection")
pygame.quit()
'@ | python -

```

All three difficulty choices now reset position, jump state, score, obstacles, timer, and distance; the settings are dis

```

Ran terminal command: $env:PYTHONDONTWRITEBYTECODE = '1'; $env:SDL_VIDEODRIVER = 'dummy'; @'
import pygame
from game.game_engine import GameEngine
from game.obstacle import Obstacle

pygame.init()
engine = GameEngine(800, 400)
engine.score = 3
engine.obstacles = [Obstacle(160, engine.ground_y, 120)]
engine.update()
assert engine.game_over, "Swept collision did not reach Game Over"
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_3))
assert not engine.game_over and engine.difficulty == "Hard"
assert engine.speed == 8 and engine.spawn_interval == 55
assert engine.score == 0 and not engine.obstacles
engine.update()
assert not engine.game_over and engine.speed > 8
assert engine.score == 0 and not engine.obstacles
print("PASS: actual collision -> Hard selection -> fresh running game")
pygame.quit()
'@ | python -

```

The end-to-end check passed from a real swept collision through selecting Hard and resuming a fresh run. I'm checking sy

```

Ran terminal command: @'
from pathlib import Path
for filename in ("main.py", "game/game_engine.py"):
    source = Path(filename).read_text(encoding="utf-8")
    compile(source, filename, "exec")
print("PASS: Python syntax/compile checks for main.py and game/game_engine.py")
'@ | python -

```

```
Ran terminal command: git diff -- main.py game/game_engine.py; git status --short
```

```
Checked , , no problems found
```

Task 3 is implemented. The Game Over screen now offers `1 - Easy`, `2 - Medium`, `3 - Hard`, and `4 - Exit`. Easy, Medium

Validation passed: an actual collision led to Game Over, selecting Hard started a fresh running game, all three settings

****Files:**** This Task 3 change modified `game\_engine.py`. `main.py` remains modified from Task 2 and provides the exit ha

Made changes.